

K I N E M A

INTERACTIVE PHYSICS WORKBENCH

Desktop Edition :: Design and Architecture Document

FIELD	VALUE
Codename	KINEMA (from cinemática, the physics of motion)
Document	Design and Architecture Document, revision 0.1 (draft)
Date	3 October 2026
Product type	Offline desktop application. Single executable. No network, no accounts.
Primary platform	Windows first. Linux and macOS builds follow from the same workspace.
Primary language	Rust (provisional, see section 7.1). C with raylib is the documented fallback.
Visual style	Old but functional: classic grey desktop widgets, bevelled panels, VGA palette, keyboard first.
Governing rules	Master Directive on Architecture, Resilience and Deterministic Software Quality (applied in sections 7 to 9).
Market ready?	No. This is an explicit non-goal. The goal is learning, with real quality discipline.

Reading guide

SECTION	WHAT IT ANSWERS
1 - 3	What KINEMA is, what it is not, and what 'old but functional' means in practice.
4 - 6	The desktop application itself: window layout, menus, shortcuts, file format, and the five simulation modules with their equations.
7	Architecture: hexagonal layers, the physical Cargo workspace, ports, concurrency, and how each clause of the directive applies to a desktop program.
8	Quality engineering: stability metrics, complexity rules, the maintainability budget, and the technical-debt model with worked numbers.
9 - 10	Delivery pipeline (commits, SonarCloud, CI) and the milestone roadmap.
11 - 12	Risks, open decisions, glossary, and the Definition of Done checklist.

1. EXECUTIVE SUMMARY

KINEMA is a desktop physics workbench in which every equation is live. A student places a vehicle on a track, types a velocity, presses play, and watches the body move while its position, velocity and acceleration graphs and the governing formulas update on the same frame. Editing any number, whether in a formula, in a slider, or by dragging a body, changes the simulation immediately.

The application covers the first-year mechanics syllabus: uniform rectilinear motion (MRU), uniformly varied motion (MRUV), free vertical motion and falling objects (MVL), dynamics with friction, and ropes and pulleys. Its signature feature is the **meeting solver**: place two bodies, and KINEMA computes, draws and verifies the instant and position at which they meet.

The project is deliberately a learning vehicle. It pairs a demanding low-level language with a strict architecture and a visible quality bar, and applies both to a domain whose answers can be checked by hand. It is not meant to be sold, only to be correct, fast, and finished.

Key properties

- **Equation-first.** Formulas are not decoration. They are the control surface. Change a number and the world changes.
- **Desktop-native and offline.** One executable, instant start, no installer, no network code, no telemetry.
- **Deterministic.** The same scene always produces the same result. Closed-form physics where it exists, a fixed-timestep integrator where it does not.
- **Verifiable.** Every numeric result is tested against an analytic answer. Every module ships with hand-checkable presets.
- **Architecturally strict.** A hexagonal core split into physical Cargo crates, so the compiler itself enforces the dependency rule.
- **Classic look.** Grey bevelled widgets, a 16-colour palette, monospace numbers, and keyboard shortcuts for everything.

2. GOALS AND NON-GOALS

2.1 Goals

ID	GOAL	HOW WE KNOW IT IS MET
G1	Cover MRU, MRUV, free vertical motion (MVL), dynamics with friction, and ropes.	Each module passes its acceptance criteria in section 6.
G2	Equation-first interaction: every number is editable and every editable number is visible.	Editing a value updates scene, graphs and formulas within one frame (16 ms).
G3	Two bodies meeting: solve, draw and verify the meeting instant and point.	Solver matches the analytic root to a relative error under $1e-9$.
G4	Desktop-native, offline, instant.	Cold start under 1 s. No network symbols in the binary.
G5	Deterministic and verifiable.	Numeric integrator results are within a stated bound of the analytic solution in tests.
G6	Learn a systems language and strict architecture.	Quality Gate PASSED from the first merged commit.

2.2 Non-goals

- **Not market ready.** No installer polish, licensing, localisation, auto-update or support burden.
- **No web or mobile build.** The architecture allows it later, but nothing is designed for it now.
- **No accounts, cloud sync, multiplayer or telemetry.**
- **No 3D.** Version 1 is 2D, and most modules are effectively 1D.
- **No general rigid-body engine.** No arbitrary polygon collisions in version 1.
- **No scripting language.** The equation panel edits parameters, not programs.

3. DESIGN PHILOSOPHY: OLD BUT FUNCTIONAL

The visual and interaction language is borrowed from the era of classic desktop software: software that started instantly, showed everything on screen at once, and could be driven entirely from the keyboard. The aim is not nostalgia for its own sake. It is that the constraints of that era produce tools that are fast, legible and honest.

PRINCIPLE	WHAT IT MEANS IN KINEMA
One file, no setup	A single executable. No installer, no runtime, no registry entries. Preferences live in a plain text file next to it.
Instant	Window visible in under one second. No splash screen. No loading bar.
Everything visible	Scene, equations, graphs and inspector share one window. No hidden tabs for core features.
Keyboard first	Every command has a shortcut and a menu entry. The mouse is optional.
Static chrome, live content	No animated widgets, no fades. The only thing that moves is the simulation.
Honest numbers	Monospace values, explicit units on every field, fixed decimal places that the user can change.
Offline by construction	No network code exists in the workspace. This also removes whole classes of failure and security work.
Deterministic	No randomness, fixed timestep, no dependence on frame rate. A saved scene replays identically.

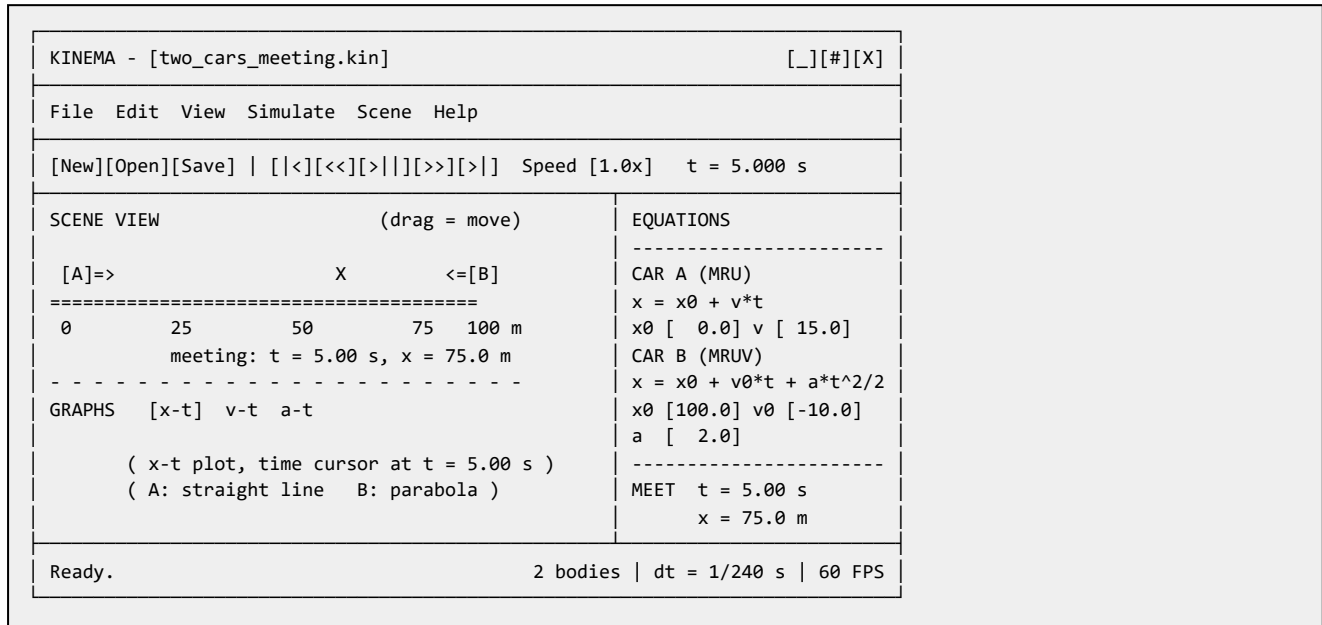
4. USERS AND SCENARIOS

USER	SCENARIO	WHAT KINEMA MUST DO
Student	Solves a textbook problem by hand, then checks it: two trains approach from 300 km apart, when do they meet?	Let the student enter the same numbers, see the meeting instant and point, and compare with their algebra.
Student	Does not trust a formula. Wants to see what happens when acceleration doubles.	Change one number and watch the graphs bend, with the formula visible beside them.
Teacher	Demonstrates that a feather and a hammer fall together on the Moon.	Gravity presets, two falling bodies, a shared time cursor and synchronous graphs.
Author	Wants a hard but achievable project in a systems language.	A strict architecture, a visible quality gate, and a feature list that grows in small, finishable milestones.

5. DESKTOP APPLICATION DESIGN

5.1 Main window

One resizable window with a classic menu bar, a toolbar, two main panels, and a status bar. The minimum supported size is 800 by 600 pixels. The layout below is the reference mock-up.



5.2 Panels

PANEL	PURPOSE	INTERACTION
Scene view	The world: track, vehicles, blocks, ropes, falling objects, vectors.	Drag a body to set its position. Drag a vector arrow to set velocity or force. Mouse wheel zooms. Middle drag pans.
Equation panel	The governing formulas of the selected body, with editable parameters.	Click a number, type, press Enter. Esc cancels. Up and down arrows nudge by the field's step.
Graph panel	x-t, v-t and a-t plots for all bodies, with a shared time cursor.	Click a graph to place the time cursor. The scene jumps to that instant.
Inspector	Constants and derived quantities: meeting time, stopping time, apex height, impact speed, tension, friction state.	Read-only except for scene constants such as gravity and friction coefficients.
Status bar	Run state, body count, timestep, frame rate, last warning.	Warnings (for example, rejected input) stay until the next successful action.

5.3 Menu structure

MENU	ENTRIES
File	New, Open..., Save, Save As..., Open Preset >, Export Image (PNG), Exit
Edit	Undo, Redo, Duplicate Body, Delete Body, Preferences...
View	Equation Panel, Graph Panel, Inspector, Vectors (velocity, acceleration, force), Grid, Trails, Theme > (Classic, Phosphor, Amber)

MENU	ENTRIES
Simulate	Play / Pause, Step Forward, Step Back, Reset, Speed >, Gravity >
Scene	Add Vehicle, Add Falling Object, Add Block, Add Surface / Incline, Add Rope, Add Pulley
Help	Contents (F1), Equation Reference, About KINEMA

5.4 Keyboard shortcuts

KEY	ACTION	KEY	ACTION
Space	Play / Pause	Ctrl+N / O / S	New / Open / Save
Right / Left	Step one frame forward / back	Ctrl+Z / Ctrl+Y	Undo / Redo
Shift+Right / Left	Jump one second forward / back	Tab	Select next body
Home	Reset time to $t = 0$	Enter / Esc	Commit / cancel field edit
+ / -	Zoom scene	F1	Help

5.5 Visual style specification

ELEMENT	SPECIFICATION
Window and panels	Face colour #C0C0C0. Raised bevel: white top-left, #808080 bottom-right. Sunken bevel for input fields.
Title bar	Solid #000080 with white bold text. No gradient.
Scene canvas	Black background, grid in #555555. VGA-style colours: Body A #FF5555, Body B #55FFFF, velocity #55FF55, acceleration #FFFF55, force #FF55FF.
Typography	A bundled bitmap-style monospace font for all numbers and formulas. A small sans-serif for menus. Font choice must have a permissive licence.
Themes	Classic (grey, default), Phosphor (#33FF33 on black), Amber (#FFB000 on black). Theme is a palette swap only.
Motion	No UI animation. Simulation redraws at up to 60 FPS. Frame pacing is independent of physics time.
Accessibility	Contrast ratio of at least 7:1 for text in the Classic theme. Every control reachable by keyboard.

5.6 Time and simulation control

- **Closed-form modules** (MRU, MRUV, free fall) evaluate position at any instant t , forward or backward. Scrubbing is exact and free.
- **Numeric modules** (dynamics with changing forces, rope) advance with a fixed timestep of 1/240 s. A ring buffer of keyframes (one every 0.5 s of simulated time, at most 600) allows scrubbing backward by deterministic replay from the nearest keyframe.
- **Speed** ranges from 0.1x to 10x. Changing speed changes how many fixed steps run per frame, never the step size.
- **Undo and redo** keep at most 200 edits. Older edits are evicted.

5.7 Scene file format (.kin)

Scenes are saved as human-readable text so that they can be diffed, edited by hand and attached to a homework answer. The format is deliberately simple and hand-parsed, with no serialisation framework.

```
# KINEMA scene file - format 1
[scene]
name    = "Two cars meeting"
gravity = 9.80665          # m/s^2

[body.car_a]
kind    = "vehicle"
motion  = "mru"
x0      = 0.0              # m
v       = 15.0             # m/s

[body.car_b]
kind    = "vehicle"
motion  = "mruv"
x0      = 100.0
v0      = -10.0
a       = 2.0              # m/s^2
```

INPUT HARDENING A .kin file is untrusted input. The reader enforces a 1 MiB size limit, at most 256 bodies, rejects NaN and infinity, treats unknown keys as warnings, and never panics on malformed text. A locked or unreadable file is reported in the status bar and the program continues with the in-memory scene.

6. SIMULATION MODULES

Modules are built in the order below. Each is complete, tested and releasable before the next begins.

6.1 Module M1: MRU (uniform rectilinear motion)

A vehicle on a straight track moves at constant velocity. The user places it, sets its velocity, and presses play.

QUANTITY	EQUATION	NOTES
Position	$x(t) = x_0 + v \cdot t$	x_0 in m, v in m/s, t in s
Velocity	$v(t) = v$	constant
Acceleration	$a(t) = 0$	shown as a flat line on the a-t graph
Meeting instant	$t^* = (x_{0B} - x_{0A}) / (v_A - v_B)$	valid when $v_A \neq v_B$

- Drag the vehicle to set x_0 . Drag the velocity arrow to set v . Or type either value.
- If $v_A = v_B$ and $x_{0A} = x_{0B}$, the bodies coincide for all time. If the velocities are equal and the positions differ, they never meet. Both cases are reported in words in the inspector.
- If $t^* < 0$, the bodies met in the past. The marker is drawn in a dimmed colour and labelled 'past'.
- **Acceptance:** changing v updates scene, graphs and formulas on the same frame. The meeting instant matches the closed form to a relative error under $1e-9$.

6.2 Module M2: MRUV (uniformly varied motion)

Adds constant acceleration. The same scene, the same controls, and the same meeting solver, which now handles a quadratic.

QUANTITY	EQUATION	NOTES
Position	$x(t) = x_0 + v_0 \cdot t + \frac{1}{2} \cdot a \cdot t^2$	
Velocity	$v(t) = v_0 + a \cdot t$	
Time-free relation	$v^2 = v_0^2 + 2 \cdot a \cdot (x - x_0)$	used for stopping distance
Stopping instant	$t_s = -v_0 / a$	only when v_0 and a have opposite signs
Meeting condition	$\frac{1}{2}(a_A - a_B)t^2 + (v_{0A} - v_{0B})t + (x_{0A} - x_{0B}) = 0$	up to two solutions

Meeting solver: cases

CASE	CONDITION	RESULT	SHOWN AS
Constant	all three coefficients ≈ 0	same trajectory, or never meet	text in inspector
Linear	leading coefficient ≈ 0	one instant, or none	one marker
No real root	discriminant < 0	bodies never meet	'no meeting' message
Double root	discriminant $= 0$	bodies touch once and separate	one tangent marker
Two roots	discriminant > 0	two instants	two markers; past ones dimmed

Worked example (the preset shown in the mock-up). Car A: MRU, $x_0 = 0$, $v = 15$ m/s. Car B: MRUV, $x_0 = 100$ m, $v_0 = -10$ m/s, $a = +2$ m/s². The condition becomes $-t^2 + 25t - 100 = 0$, that is $t^2 - 25t + 100 = 0$, with roots $t = 5$ s ($x = 75$ m) and $t = 20$ s ($x = 300$ m). Car B turns around at $t = 5$ s, and Car A passes it again at $t = 20$ s. This preset is a regression test.

- **Acceptance:** both roots are found, ordered, and drawn on scene and graph. Near-zero leading coefficients fall back to the linear case without precision loss.

6.3 Module M3: MVL (free vertical motion and falling objects)

Vertical motion under gravity: dropped, thrown up, or thrown down. Gravity is a scene constant with presets for the Earth (9.81 m/s²), the Moon (about 1.62), Mars (about 3.71) and Jupiter (about 24.79), or any custom value.

QUANTITY	EQUATION	NOTES
Height	$y(t) = y_0 + v_0 \cdot t - \frac{1}{2} \cdot g \cdot t^2$	upward is positive
Velocity	$v(t) = v_0 - g \cdot t$	
Time to apex	$t_{up} = v_0 / g$	when $v_0 > 0$
Maximum height	$h_{max} = y_0 + v_0^2 / (2g)$	
Impact instant	$t_i = (v_0 + \sqrt{v_0^2 + 2 \cdot g \cdot y_0}) / g$	ground at $y = 0$
Impact speed	$ v_i = \sqrt{v_0^2 + 2 \cdot g \cdot y_0}$	

- Several objects can fall together. Without air resistance, mass does not change the motion, which makes a 'feather and hammer on the Moon' preset a strong demonstration.
- Apex, impact instant and impact speed appear in the inspector and as markers on the graphs.
- **Acceptance:** with $y_0 = 20$ m, $v_0 = 0$, $g = 9.81$ m/s², the impact instant is about 2.019 s and the impact speed about 19.8 m/s.

6.4 Module M4: dynamics with friction

A block on a horizontal surface or an incline, with an applied force, mass, friction coefficients and angle all editable. A free-body diagram shows every force as a labelled arrow whose length is proportional to its magnitude.

QUANTITY	EQUATION	NOTES
Newton's second law	$\Sigma F = m \cdot a$	along the direction of motion
Weight and normal force	$W = m \cdot g$; $N = m \cdot g \cdot \cos \theta$	$N = m \cdot g$ on the flat
Static friction limit	$f_s \leq \mu_s \cdot N$	block stays at rest while the net driving force is below this limit
Kinetic friction	$f_k = \mu_k \cdot N$	acts opposite to motion
Horizontal, applied force F	$a = (F - \mu_k \cdot N) / m$	once $ F > \mu_s \cdot N$
Incline, sliding down	$a = g \cdot (\sin \theta - \mu_k \cdot \cos \theta)$	slides if $\tan \theta > \mu_s$

- The inspector shows the friction state (STATIC or KINETIC) and the current net force.
- Constant-force regimes use the closed form. Changing forces use semi-implicit Euler at 1/240 s.
- **Acceptance:** the block stays at rest until the applied force passes the static limit, then accelerates at the analytic rate to within the integrator's stated error bound.

6.5 Module M5: rope and pulleys

The hardest module, built in two stages so that each is testable.

Stage A: ideal rope (massless, inextensible)

SYSTEM	EQUATIONS
Atwood machine (two hanging masses)	$a = (m_2 - m_1) \cdot g / (m_1 + m_2)$; $T = 2 \cdot m_1 \cdot m_2 \cdot g / (m_1 + m_2)$
Block on table pulled by hanging mass, with friction	$a = (m_2 \cdot g - \mu_k \cdot m_1 \cdot g) / (m_1 + m_2)$; $T = m_1 \cdot (a + \mu_k \cdot g)$

Stage B: particle-chain rope

The rope becomes N point masses (24 by default) joined by distance constraints, advanced with Verlet integration and a fixed number of constraint-relaxation passes (12 by default). The user can pull an end node with a chosen force, drag the rope across a surface with Coulomb friction on each node, and change gravity, total mass and node count. Tension is estimated from the constraint corrections and drawn as a colour ramp along the rope.

- **Acceptance (Stage A):** simulated acceleration and tension match the formulas above within the integrator bound.
- **Acceptance (Stage B):** stretch stays under 1% at default tension, the simulation remains stable for 10 minutes of wall-clock time, and no value ever becomes NaN or infinite.

6.6 Stretch modules

- Projectile motion in 2D with launch angle and optional air resistance.
- Springs and simple harmonic motion with phase-space plot.
- One-dimensional elastic and inelastic collisions.
- 'Challenge mode': the program states a problem and the user must set the right values to satisfy it.

7. ARCHITECTURE

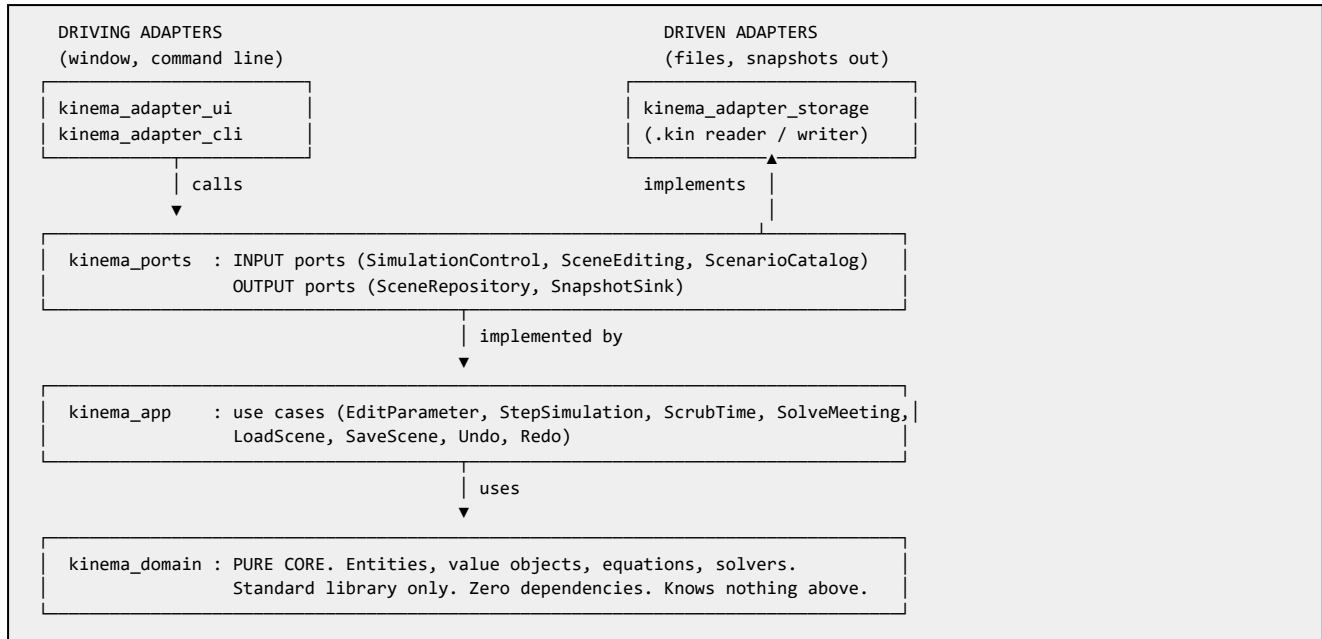
KINEMA follows the Master Directive: a hexagonal core, physically split into separate packages, where outer layers depend on inner layers and the core knows nothing about the outside world. For a desktop program, the 'outside world' is the window, the file system and the command line.

7.1 Language decision

OPTION	FIT WITH THE DIRECTIVE	FIT WITH THE GOAL
Rust (primary)	Cargo workspaces map one-to-one to hexagonal packages. A crate cannot use what its manifest does not list, so the compiler enforces the dependency rule.	A real challenge (ownership, lifetimes) that is still achievable. Strong tooling for tests, coverage and linting. SonarCloud analyses Rust.
C with raylib + raygui	Possible with separate static libraries and CMake targets, but the dependency rule is enforced by discipline, not by the toolchain.	The most hands-on, minimal option. More manual work for the same features.
C++	CMake targets give physical separation. A calibration row exists in the directive.	Comfortable, but the least 'new'.
HolyC / Assembly	Cannot satisfy workspaces, coverage or CI.	Fun experiments, not suitable for this scope.

NOTE The directive's calibration matrix (section 4.4) lists Dart, Go, C++, JavaScript/TypeScript, PHP and C, but not Rust. Section 8.5 therefore uses clearly labelled provisional constants for Rust, to be recalibrated from real data after milestone M3.

7.2 Layers and the dependency rule



Dependency rule: arrows point inward only. The domain depends on nothing. Ports depend on the domain. The application layer depends on the domain and ports. Adapters depend on ports and the domain, and never on each other or on the application layer. Only the composition root, the binary crate, sees everything and wires it together.

PRESENTER PORT The window must receive new state, so the UI adapter also implements the output port `SnapshotSink`. This is the standard 'presenter' arrangement. The domain still never learns that a window exists.

7.3 Physical workspace layout

```

kinema/
├─ Cargo.toml                # [workspace]; shared versions; internal deps by relative path
├─ clippy.toml
├─ sonar-project.properties
├─ .github/workflows/ci-sonarcloud.yml
├─ apps/
│   └─ kinema/                # composition root (IoC container), the only binary
│       └─ src/main.rs
└─ crates/
    ├─ kinema_domain/         # pure core: entities, equations, solvers
    ├─ kinema_ports/          # contracts: input and output port traits
    ├─ kinema_app/            # use cases / orchestration
    ├─ kinema_adapter_ui/     # driving: desktop window and widgets
    ├─ kinema_adapter_cli/    # driving: headless runner for batch checks and tests
    └─ kinema_adapter_storage/# driven: .kin reader and writer
  
```

CRATE	LAYER	INTERNAL DEPENDENCIES	EXTERNAL DEPENDENCIES
kinema_domain	Domain	none	none (standard library only)
kinema_ports	Ports	domain	none
kinema_app	Application	domain, ports	none
kinema_adapter_ui	Driving adapter	domain, ports	UI toolkit (provisional: egui / eframe)

CRATE	LAYER	INTERNAL DEPENDENCIES	EXTERNAL DEPENDENCIES
kinema_adapter_cli	Driving adapter	domain, ports	none
kinema_adapter_storage	Driven adapter	domain, ports	none (hand-written, size-limited parser)
apps/kinema	Composition root	app, ports, ui, cli, storage	none

Internal dependencies are declared once in the workspace manifest as relative paths and inherited by each crate. This makes circular coupling impossible: Cargo refuses to build a cycle.

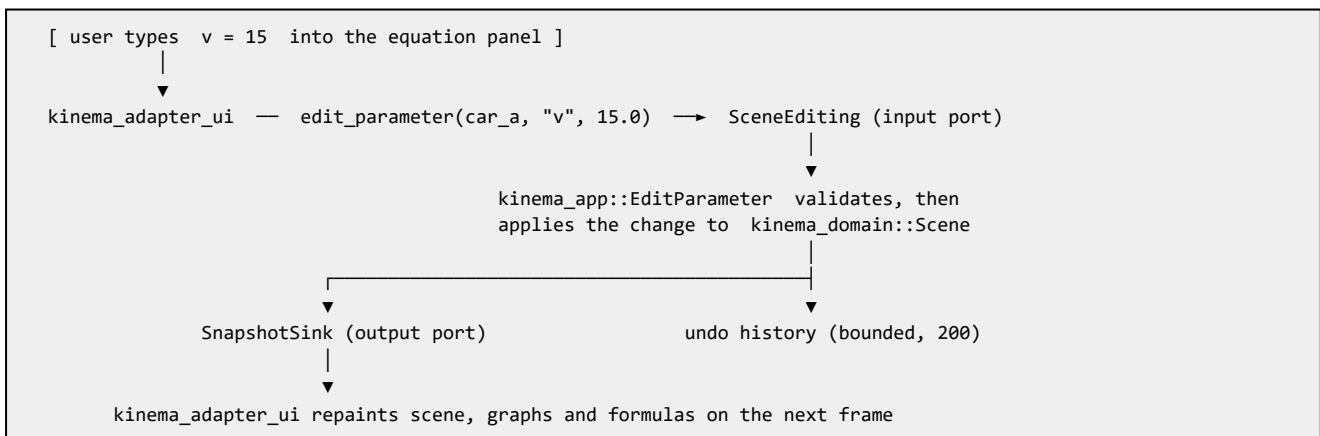
7.4 Ports

PORT	DIRECTION	RESPONSIBILITY
SimulationControl	Input	play, pause, step, reset, seek(t), set_speed
SceneEditing	Input	add_body, remove_body, edit_parameter(body, name, value), undo, redo
ScenarioCatalog	Input	list and load built-in presets such as 'two cars meeting'
SceneRepository	Output	load(path) and save(path, scene). Implemented by the storage adapter.
SnapshotSink	Output	receives an immutable snapshot of the world after each change. Implemented by the UI adapter.

Domain abstractions

To keep the domain from becoming a pile of concrete types (see the abstraction target in section 8.1), polymorphism is built into the core: `Motion1D` (position, velocity and acceleration at time t , implemented by MRU, MRUV and free fall), `Integrator`, `ForceLaw`, `Constraint`, `FrictionModel`, and `ParametricLaw`. The last one exposes a body's editable parameters and its formula text, so the equation panel can render any module without special cases.

7.5 Data flow of one edit



7.6 Concurrency, memory and cancellation

A desktop program has no servers, but it still has two threads and several unbounded-growth risks. The directive's resilience rules are applied as follows.

CONCERN	RULE IN KINEMA
Workers	Exactly one simulation worker thread (MaxConcurrentWorkers = 1) plus the UI thread.
Command queue	UI to simulation commands pass through a bounded channel (MaxQueueSize = 64). If it is full, the command is rejected immediately and the status bar says 'busy, input dropped'. No unbounded queues anywhere.
Snapshots to UI	A single-slot 'latest value' mailbox. The UI always renders the newest state and never accumulates a backlog.
Graph history	Fixed-capacity ring buffer (20 000 samples per series). Oldest samples are evicted.
Keyframes and undo	Fixed caps (600 keyframes, 200 undo steps) with oldest-first eviction. Memory use stays flat over long sessions.
Cancellation	Long operations (PNG export, headless batch runs) take a cancellation token. On cancel they stop, close handles, and delete any partial temporary files.
Resource locks	If a preferences or scene file is locked, the error is caught, the resource is skipped, and the program continues in memory with a visible warning. It never panics.
Event-driven display	The simulation pushes snapshots on change and the UI repaints on events, targeting 60 FPS. There is no busy-wait polling loop.

7.7 How each clause of the directive applies

Several clauses of the directive are written for networked services. KINEMA has none, so those clauses are marked 'not applicable' with the reason, and the closest meaningful equivalent is applied where one exists.

DIRECTIVE SECTION	APPLIES?	KINEMA TREATMENT
1. Hexagonal architecture, physical workspaces	Yes	Sections 7.2 to 7.4.
2. Package metrics (Ca, Ce, I, A, D)	Yes	Targets and calculations in section 8.1. Every crate must satisfy $D < 0.70$.
3. $CC \leq 5$, $MI \geq 75$, why-only comments	Yes	Sections 8.2 to 8.4.
4. Technical-debt financial model	Yes, adapted	Used as a budgeting aid. Money is replaced by hours. Section 8.5.
5. Circuit breaker	Not applicable	There are no external calls. If a network feature is ever added, it must ship with a circuit breaker first.
5. Backpressure, eviction, cancellation	Yes	Section 7.6.
6. Silent subprocesses, process-tree kill	Not applicable in v1	No subprocesses are spawned. If one is added (for example, an external encoder), it must suppress its console window and be killed as a tree on exit.
6. Degrade on locked resources	Yes	Section 7.6, 'Resource locks'.
7. CORS, credentials, parametrised queries	Not applicable	No server, no credentials, no SQL, no shell commands built from text. The equivalent risk is untrusted .kin files, hardened as described in section 5.7.
8. Streaming instead of polling	Yes, adapted	Event-driven snapshots from the worker to the UI, with no polling loops.
9. Conventional commits, SonarCloud, CI	Yes	Section 9.
10. Rejection checklist	Yes	Adopted as the Definition of Done in Appendix A.

8. QUALITY ENGINEERING

8.1 Package stability metrics (design targets)

Using the directive's definitions: $I = Ce / (Ca + Ce)$; $A = Na / Nc$; $D = |A + I - 1|$. Dependencies are counted as unique packages, never as import lines. The table below is the design target for the planned crate graph, and is recomputed at every milestone.

CRATE	Ca	Ce	I	A (target)	D	RESULT
kinema_domain	5	0	0.00	0.40 (6 of 15 types)	0.60	pass, tightest margin
kinema_ports	4	1	0.20	0.80 (8 of 10)	0.00	pass
kinema_app	1	2	0.67	0.13 (1 of 8)	0.21	pass
kinema_adapter_ui	1	2	0.67	0.00	0.33	pass
kinema_adapter_cli	1	2	0.67	0.00	0.33	pass
kinema_adapter_storage	1	2	0.67	0.00	0.33	pass
apps/kinema	0	5	1.00	0.00	0.00	pass

WHY THE DOMAIN NEEDS ABSTRACTIONS The domain is maximally stable ($I = 0$), so the rule $D < 0.70$ forces A above 0.30. A domain made only of concrete structs would sit in the 'zone of pain' and be rejected. The six domain traits listed in section 7.4 are therefore a design requirement, and they also make the modules interchangeable. The target is $A = 0.40$ to leave a margin.

8.2 Complexity rule: $CC \leq 5$ in practice

Cyclomatic complexity is branch points plus one. Every function must stay at 5 or below, and anything above is split into small pure helpers. Physics code invites long case analyses, so the meeting solver is designed around this rule from the start.

```
// kinema_domain/src/meeting.rs
pub fn meeting_times(q: Quadratic) -> Roots {
    match q.degree() {
        Degree::Constant => solve_constant(q),
        Degree::Linear    => solve_linear(q),
        Degree::Quadratic => solve_quadratic(q),
    }
}

pub fn solve_quadratic(q: Quadratic) -> Roots {
    match q.discriminant().partial_cmp(&0.0) { // CC = 4
        Some(Ordering::Less)    => Roots::None,
        Some(Ordering::Equal)   => Roots::One(q.vertex_t()),
        Some(Ordering::Greater) => Roots::Two(stable_pair(q)),
        None                    => Roots::None, // NaN input
    }
}

// Why: the textbook formula cancels badly when b^2 is much larger than 4ac.
// Compute q = -(b + sign(b) * sqrt(D)) / 2 and derive both roots from it.
fn stable_pair(q: Quadratic) -> (f64, f64) { /* ... */ }
```

- Prefer guard clauses and early returns over nested conditions.
- Prefer table-driven or trait-driven dispatch (for example, `Motion1D`) over long `match` chains.
- Each helper has one job and a name that says it. A helper that cannot be named clearly is two helpers.

8.3 Maintainability budget ($MI \geq 75$)

The directive defines $MI = \max(0, (171 - 5.2 \cdot \ln(LOC) - 0.23 \cdot CC) / 171 \times 100)$, where LOC counts logical executable lines. Solving the formula gives a practical size budget per measured unit. The values below assume the worst allowed complexity, $CC = 5$.

LOGICAL LINES (LOC)	50	110	500	1 000	2 900
MI at CC = 5	87.4	85.0	80.4	78.3	75.1

- **Gold standard ($MI \geq 85$):** keep each unit at about 110 logical lines or fewer.
- **Hard ceiling ($MI \geq 75$):** about 2 900 logical lines at $CC = 5$. Beyond that the unit must be split.
- In practice: small modules, one idea per file.

8.4 Comment policy

Comments never narrate syntax. Identifiers carry the 'what'. Comments carry only the 'why': a trade-off, a numerical hazard, a platform constraint, or a counter-intuitive decision. Typical examples in this codebase are the fixed timestep choice, the stable quadratic formula above, and the reason a keyframe interval is 0.5 s.

8.5 Technical-debt model (budgeting aid)

The directive treats code as an asset that accrues debt. For a solo project the model is used to decide when refactoring is worth the time, with the cost unit set to one hour ($C_{\text{hora}} = 1$).

Friction factor

$F(MI) = 1 + ((75 - MI) / 75) \cdot 3$, with a ceiling of 4.0 at $MI = 0$. Using the directive's frontend base time of 4 hours:

MI	75	60	45	30	0
Friction factor F	1.0	1.6	2.2	2.8	4.0
Real time for a 4 h task	4.0 h	6.4 h	8.8 h	11.2 h	16.0 h

Provisional constants for Rust

CONSTANT	VALUE	BASIS
CC threshold	5	The directive's global rule ($CC \leq 5$).
K (hours per excess CC point)	0.60 h	Provisional. Between the C and Go rows.
CDU (hours per LOC)	0.45 h	Provisional. Between the C and Go rows.
$FF_{\text{base}} / FF_{\text{max}}$	0.12 / 0.30	Copied from the C row as the nearest low-level analogue.
α_{base}	1.0	Assumed. The directive does not give a value.
C_{hora}	1.0	One hour equals one unit, since there is no payroll.

Worked example (hypothetical, to show how the numbers behave). Suppose `kinema_domain` has 2 000 logical lines and 6 points of excess complexity. Asset value $VA = 2\,000 \times 0.45 \times 1 = 900$ h. Repair cost $CR = 6 \times 0.60 \times 1 = 3.6$ h. Debt ratio $TDR = 3.6 / 900 = 0.4\%$, well under the 5% limit. With $Ca = 5$, $FF = 0.12 + 0.30 \times 5/6 = 0.37$. With $I = 0$ and $A = 0.40$, $\alpha = 1 + (1 - 0)(1 - 0.40) = 1.6$ and $D = 0.60$. Opportunity cost $CO = 3.6 \times (1.6 - 1) \times (1 + 0.60) = 3.456$ h. Cost of doing nothing $CNHN = 3.6 \times 0.37 + 3.456 = 4.788$ h. $ROI = 4.788 / 3.6 = 133\%$. Any ROI above 100% means the refactor pays for itself, so it is scheduled.

RECALIBRATE All Rust constants above are assumptions. After milestone M3, replace them with measured values (actual hours per refactor, actual lines per hour) and update this section.

8.6 Testing strategy

- **Analytic reference tests.** Every closed-form module is tested against hand-computed values (for example, the two-root meeting preset and the 20 m drop).
- **Numeric versus analytic.** The integrator is run on cases with known solutions, and the error must stay within a stated bound.
- **Property tests.** Randomised inputs check invariants: energy bookkeeping in free fall, symmetry of the meeting solver, no NaN output for any finite input.
- **Parser tests.** Malformed, oversized and adversarial .kin files must be rejected cleanly.
- **Coverage.** At least 80% on new code, measured with `cargo llvm-cov` and exported as LCOV for SonarCloud. UI adapter logic is kept in testable view-model code so that only thin rendering calls are untested.

9. DELIVERY PIPELINE

9.1 Commits

Every commit is one atomic, passing unit of change in the Conventional Commits format `<type> (<scope>) : <summary>`, with types feat, fix, refactor, perf, test and ci. Structural refactors and bug fixes are never mixed in one commit. Examples:

```
feat(domain): add MRU motion with closed-form position
test(domain): cover two-root meeting preset (t = 5 s and t = 20 s)
refactor(domain): split solve_quadratic to keep CC <= 5
feat(ui): bind equation panel fields to edit_parameter
ci: fail the build when clippy reports warnings
```

9.2 SonarCloud Quality Gate (non-negotiable)

METRIC	THRESHOLD
Coverage on new code	≥ 80.0%
Duplicated lines	< 3.0%
Reliability rating	A (0 open bugs)
Security rating	A (0 vulnerabilities, 0 unreviewed hotspots)
Maintainability rating	A (technical-debt ratio < 5.0%)
Function complexity	No new function above CC = 5

SonarCloud analyses Rust through Cargo and Clippy, so CI-based analysis is required and both tools must be installed on the runner. Coverage is imported from LCOV through the Rust-specific report property.

9.3 sonar-project.properties

```
sonar.projectKey=your-org_kinema
sonar.organization=your-sonarcloud-org
sonar.host.url=https://sonarcloud.io

sonar.sources=crates,apps
sonar.test.inclusions=**/tests/**
sonar.sourceEncoding=UTF-8

sonar.exclusions=**/target/**
sonar.coverage.exclusions=apps/kinema/src/main.rs

sonar.rust.lcov.reportPaths=lcov.info
```

9.4 CI workflow (.github/workflows/ci-sonarcloud.yml)

```
name: CI and SonarCloud Quality Gate
on:
  push:
    branches: [ "main" ]
  pull_request:
    types: [ opened, synchronize, reopened ]

jobs:
  quality:
    name: Build, test, analyse
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0          # full history for accurate analysis
      - uses: dtolnay/rust-toolchain@stable
        with:
          components: clippy, llvm-tools-preview
      - uses: taiki-e/install-action@cargo-llvm-cov
      - name: Format and lint
        run: |
          cargo fmt --all --check
          cargo clippy --workspace --all-targets -- -D warnings
      - name: Complexity gate (CC <= 5)
        run: ./scripts/check_complexity.sh
      - name: Tests with coverage
        run: cargo llvm-cov --workspace --lcov --output-path lcov.info
      - name: SonarCloud scan
        uses: SonarSource/sonarcloud-github-action@master
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
          SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
      - name: Quality Gate verification
        uses: SonarSource/sonarqube-quality-gate-action@master
        timeout-minutes: 5
        env:
          SONAR_TOKEN: ${ secrets.SONAR_TOKEN }

  windows-build:
    name: Windows release build
    runs-on: windows-latest
    steps:
      - uses: actions/checkout@v4
      - uses: dtolnay/rust-toolchain@stable
      - run: cargo build --release --workspace
      - uses: actions/upload-artifact@v4
        with:
          name: kinema-windows
          path: target/release/kinema.exe
```

BEFORE USING The workflow reuses the action names from the directive. Pin them to release tags instead of 'master', and check whether SonarSource has replaced them with newer scan actions. The complexity script is a placeholder: it can wrap a complexity-metrics tool or a Clippy complexity lint as a fast local approximation of the Sonar metric.

10. ROADMAP AND MILESTONES

Every milestone ends with a green Quality Gate. No milestone starts until the previous one is merged.

MILESTONE	SCOPE	EXIT CRITERIA
M0: Skeleton	Workspace with all six crates, CI, SonarCloud wired, empty window opens, one test per crate.	Quality Gate PASSED. Dependency graph matches section 7.3.
M1: MRU	MRU module, meeting solver (linear case), equation panel, x-t graph, preset 'two cars'.	Meeting instant within 1e-9 of the closed form. Edits update everything in one frame.
M2: MRUV	Acceleration, quadratic meeting solver, v-t and a-t graphs, stopping-time marker.	Two-root preset ($t = 5$ s and 20 s) passes as a regression test.
M3: MVL and control	Free fall, gravity presets, exact scrubbing, bounded undo and redo, .kin save and load.	20 m drop gives 2.019 s and 19.8 m/s. Parser survives adversarial files. Recalibrate section 8.5.
M4: Dynamics	Block, surface, incline, friction, free-body diagram, fixed-timestep integrator, keyframe replay.	Static-to-kinetic threshold behaves as specified. Integrator error within its bound.
M5: Rope	Stage A (ideal rope, Atwood), then Stage B (particle chain with friction and applied force).	Stage B stable for 10 minutes. No NaN under fuzzed input.
M6: Polish	Themes, preset library, help system, PNG export with cancellation, release build.	Definition of Done (Appendix A) fully satisfied.

Performance targets (targets, not gates)

- Cold start under 1 second on a mid-range laptop.
- 60 FPS with 100 bodies, or a 64-node rope, on integrated graphics.
- Flat memory use over an eight-hour session.
- Release executable small enough to email.

11. RISKS AND OPEN DECISIONS

11.1 Risks

RISK	LIKELIHOOD	IMPACT	MITIGATION
Rope instability or jitter	High	Medium	Build the ideal rope first. Keep fixed timestep. Add stretch and NaN assertions in tests.
Scope creep (more physics)	High	Medium	Milestones are closed. New modules go to the stretch list.
Rust learning curve slows M0 to M1	Medium	Medium	Domain crate uses plain data and functions. Postpone advanced lifetimes and generics.

RISK	LIKELIHOOD	IMPACT	MITIGATION
UI toolkit does not suit the classic look	Medium	Low	The toolkit lives behind the UI adapter, so it can be swapped without touching the core.
Strict CC ≤ 5 slows numerical code	Medium	Low	Use trait dispatch and small pure helpers. Treat a hard-to-split function as a design smell.
Floating-point results differ across platforms	Low	Medium	Avoid platform-specific maths. Compare with tolerances, not exact equality.

11.2 Decisions still open

1. Confirm Rust as the primary language, or choose C with raylib.
2. Confirm the UI toolkit (egui / eframe is the provisional choice) after a one-day spike to test the classic look.
3. Create the SonarCloud organisation and project, and add the SONAR_TOKEN secret.
4. Choose and licence the bundled bitmap-style font.
5. Decide whether Linux and macOS builds are built in CI from M0 or only from M6.

12. GLOSSARY

TERM	MEANING
MRU	Movimiento Rectilíneo Uniforme. Straight-line motion at constant velocity.
MRUV	Movimiento Rectilíneo Uniformemente Variado. Straight-line motion at constant acceleration.
MVL	Movimiento Vertical Libre. Free vertical motion: dropped or thrown objects under gravity.
Meeting solver	The routine that finds when and where two bodies occupy the same position.
Port	An abstract contract that the core defines and the outside world implements or calls.
Adapter	Code that translates between the outside world (window, files) and a port.
Composition root	The single place that creates concrete objects and connects them to ports.
Ca / Ce	Afferent coupling (who depends on me) and efferent coupling (what I depend on).
I, A, D	Instability, abstractness and distance from the main sequence.
CC / CCE	Cyclomatic complexity, and the amount by which it exceeds the allowed threshold.
MI	Maintainability Index.
TDR	Technical-debt ratio: repair cost divided by replacement cost.
Keyframe	A saved simulation state used to replay deterministically when scrubbing backward.

APPENDIX A. DEFINITION OF DONE

A change is rejected without review if any line below fails. This list adapts the directive's rejection checklist to KINEMA.

	CHECK
[]	Coupling leak: kinema_domain imports nothing but the standard library. No adapter depends on another adapter.

	CHECK
[]	Complexity: no function above CC = 5. No unit with MI below 75.
[]	Structure: every crate has D < 0.70 after the change.
[]	Unbounded growth: no queue, buffer, history or cache without a fixed upper limit and an eviction rule.
[]	Silent failure: no panic on bad input, locked files or malformed scenes. Errors reach the status bar.
[]	Subprocesses: none spawned, or each one is silent and killed as a tree.
[]	Network: no network code added.
[]	Commits: atomic, in Conventional Commit format.
[]	Quality Gate: SonarCloud shows PASSED for coverage, bugs, duplication, security and maintainability.
[]	Numbers: any new equation has an analytic reference test.